Case Study 1: University Scholarship Allocation

Activity sheet case study #1 | Concepts: Variables, data types, operators, conditional statements, loops, lists, tuples, tuple unpacking, algorithms (bubble sort)

1. Problem Statement

A university offers merit-cum-means scholarships. Each applicant has subject-wise marks, an attendance percentage and a family income. Design a Python program that checks every applicant against the eligibility criteria (aggregate marks, subject-wise minimum, attendance and income), explains every rejection, ranks the eligible students and resolves ties using predefined rules.

2. Objectives

- Apply conditional statements to evaluate several eligibility criteria together.
- Use loops and tuple unpacking to process a list of applicant records.
- Rank eligible students by sorting tuples with a bubble-sort loop.
- Resolve equal aggregates through ordered tie-breaking rules and report which rule decided the tie.

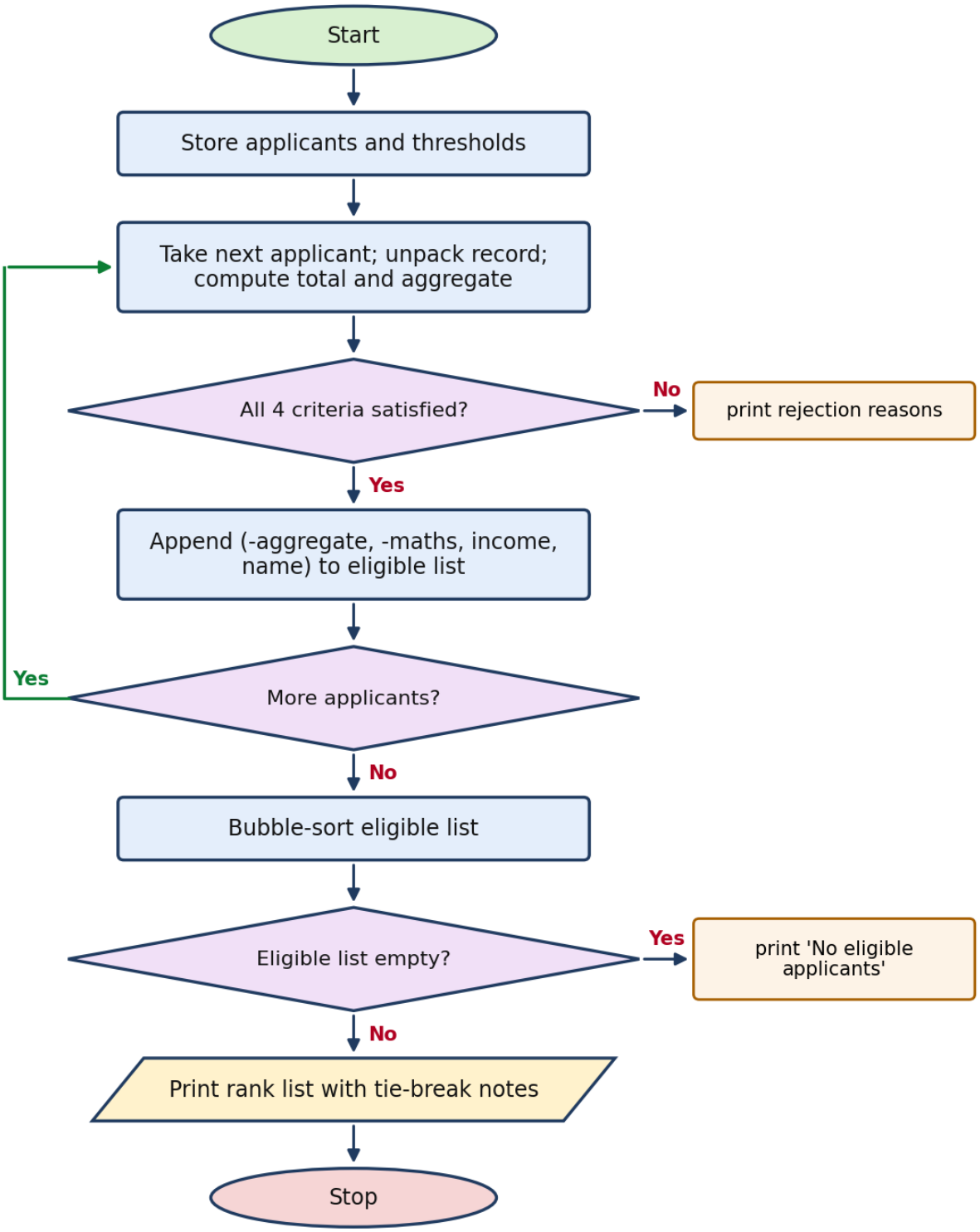
3. Assumptions

- Each applicant is a tuple: (name, [subject marks], attendance %, annual family income). The first subject is Maths.
- Eligibility thresholds: aggregate (average) of at least 75, every subject at least 50, attendance at least 80 %, income at most Rs 300000 (boundary values are eligible).
- Tie-breaking order for equal aggregates: higher Maths marks, then lower family income, then alphabetical name.
- All marks are out of 100 and the data is supplied inside the program.

4. Algorithm

1. Start and store the applicant records and the four threshold constants.
2. For each applicant, unpack name, marks, attendance and income.
3. Add all marks, note whether any subject is below the minimum, and compute the aggregate.
4. Collect a reason for each failed criterion into a list.
5. If the reason list is empty, append the key tuple (-aggregate, -maths, income, name) to the eligible list; otherwise print the reasons.
6. Sort the eligible list with bubble sort (the smaller tuple is the better applicant).
7. If nobody is eligible, print a message and stop.
8. Print the rank list; for equal aggregates print which tie-break rule decided the order.
9. Stop.

5. Flowchart



6. Pseudocode

```
START
  SET MIN_AGG=75, MIN_SUBJECT=50, MIN_ATT=80, MAX_INCOME=300000
  eligible <- empty list
  FOR each applicant (name, marks, att, income)
      total <- sum of marks ; aggregate <- total / count
      reasons <- empty list
      IF aggregate < MIN_AGG    THEN add reason
      IF any mark < MIN_SUBJECT THEN add reason
```

```
        IF att < MIN_ATT          THEN add reason
        IF income > MAX_INCOME    THEN add reason
        IF reasons is empty THEN
            append (-aggregate, -marks[0], income, name) to eligible
        ELSE
            PRINT name and reasons
    END FOR
    BUBBLE SORT eligible in ascending order of tuple
    IF eligible is empty THEN PRINT "No eligible applicants"
    ELSE
        FOR i = 0 .. n-1
            PRINT rank i+1, name, aggregate, maths, income
            IF aggregate equals previous aggregate THEN
                PRINT the tie-break rule that separated them
    END FOR
  STOP
```

7. Python Source Code

```
# Case Study 1: University Scholarship Allocation
# ---- INPUT DATA ----
# (name, [Maths, Physics, Chemistry], attendance %, annual income)
applicants = [
    ("Asha",   [92, 88, 90], 91, 250000),
    ("Bharat", [85, 91, 88], 85, 180000),
    ("Chitra", [92, 88, 90], 95, 150000),   # ties with Asha
    ("Dev",    [95, 96, 40], 90, 100000),   # fails subject minimum
    ("Esha",   [80, 82, 79], 70, 120000),   # low attendance
    ("Farhan", [90, 90, 90], 88, 450000),   # income too high
    ("Gita",   [70, 72, 68], 92, 90000),    # low aggregate
]
# ---- END DATA ----

MIN_AGGREGATE = 75.0
MIN_SUBJECT = 50
MIN_ATTENDANCE = 80
MAX_INCOME = 300000

eligible = []   # entries: (-aggregate, -maths, income, name)
print("ELIGIBILITY CHECK")
print("-" * 64)
for record in applicants:
    name, marks, attendance, income = record      # tuple unpacking
    total = 0
    weak_subject = False
    for m in marks:
        total = total + m
        if m < MIN_SUBJECT:
            weak_subject = True
    aggregate = total / len(marks)

    reasons = []
    if aggregate < MIN_AGGREGATE:
        reasons.append("aggregate below " + str(MIN_AGGREGATE))
    if weak_subject:
        reasons.append("a subject below " + str(MIN_SUBJECT))
    if attendance < MIN_ATTENDANCE:
        reasons.append("attendance below " + str(MIN_ATTENDANCE))
    if income > MAX_INCOME:
        reasons.append("income above " + str(MAX_INCOME))

    if len(reasons) == 0:
        eligible.append((-aggregate, -marks[0], income, name))
        print(name, "-> ELIGIBLE, aggregate =", round(aggregate, 2))
    else:
        print(name, "-> REJECTED:", ", ".join(reasons))

# Bubble sort: the smaller tuple is the better applicant
n = len(eligible)
for i in range(n - 1):
    for j in range(n - 1 - i):
        if eligible[j] > eligible[j + 1]:
            temp = eligible[j]
            eligible[j] = eligible[j + 1]
            eligible[j + 1] = temp

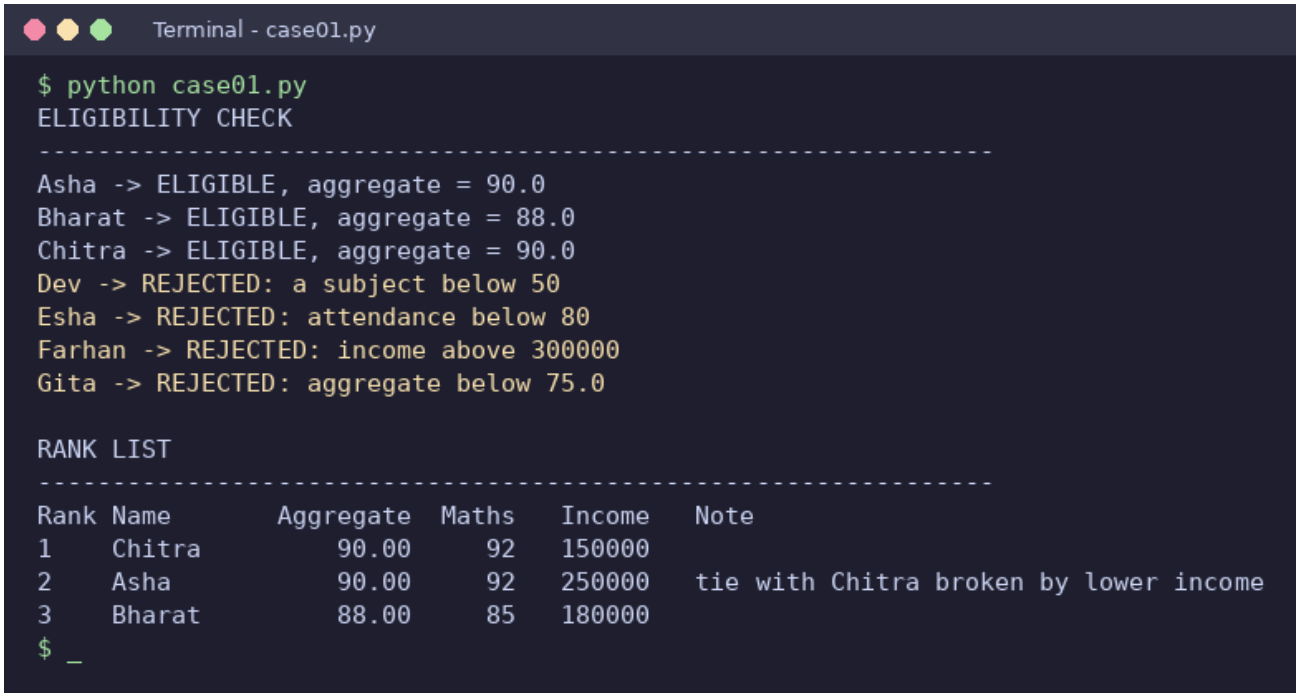
print()
print("RANK LIST")
print("-" * 64)
if n == 0:
    print("No eligible applicants.")
else:
    print("{:<5}{:<10}{:>10}{:>7}{:>9}   {}".format(
          "Rank", "Name", "Aggregate", "Maths", "Income", "Note"))
    for i in range(n):
        neg_agg, neg_maths, income, name = eligible[i]
        note = ""
        if i > 0:
            p_agg, p_maths, p_income, p_name = eligible[i - 1]
            if p_agg == neg_agg:
                if p_maths != neg_maths:
                    note = "tie with " + p_name + " broken by Maths"
                elif p_income != income:
                    note = "tie with " + p_name + " broken by lower income"
                else:
                    note = "tie with " + p_name + " broken by name"
        print("{:<5}{:<10}{:>10.2f}{:>7}{:>9}   {}".format(
              i + 1, name, -neg_agg, -neg_maths, income, note))
```

8. Test Cases

#	Test case	Input / data	Expected result	Observed output (actual)	Status
1	Default data: mixed applicants with an aggregate tie	7 applicants (see source)	Chitra ranked above Asha; Dev rejected for weak subject	`2 Asha 90.00 92 250000 tie with Chitra broken by lower income | Dev -> REJECTED: a subject below 50`	**PASS**
2	Boundary values (exactly at every threshold)	[75,75,75], att 80, income 300000	Applicant on every boundary is eligible	`Bo -> ELIGIBLE, aggregate = 75.0`	**PASS**
3	Nobody qualifies	[40,50,60], att 90, income 1000	Rejection message and empty rank list	`No eligible applicants.`	**PASS**
4	Empty applicant list	applicants = []	Program does not crash	`No eligible applicants.`	**PASS**
5	Tie broken by Maths marks	two applicants, aggregate 80 each	P (Maths 90) ranks first	`2 Q 80.00 80 100000 tie with P broken by Maths`	**PASS**
6	Several rejection reasons together	[40,40,40], att 50, income 900000	All four reasons are listed	`Mix -> REJECTED: aggregate below 75.0, a subject below 50, attendance below 80, income ...`	**PASS**

9. Output Screenshot

Terminal output of `python case01.py` using the default data:



10. Conclusion

The program evaluates every applicant against all four criteria, lists each reason for rejection, ranks the eligible students and records which tie-break rule decided equal aggregates. It demonstrates conditional logic, loops, lists, tuples, tuple unpacking and a manual sorting algorithm without any advanced features.